1 APIOBike

1.1 Subtask 1

In this subtask, the road network is a path with stations numbered $0, 1, ..., N-1$ in sequential order. Every station i satisfies $A[i] \neq B[i]$, and there is exactly one station s that initially contains all bikes.

Our goal is to redistribute bikes from station s to all other stations. The rebalancing truck starts at s and loads $A[s] - B[s]$ bikes. The number of bikes on the truck then equals $\sum_{i \neq s} B[i]$. As the truck visits each station $i \neq s$, the driver unloads exactly $B[i]$ bikes. To complete the rebalancing, the truck must visit every station at least once.

To minimize the travelling distance while visiting all stations, we consider two candidate walks:

- $X = [s, s+1, ..., N-2, N-1, N-2, ..., 0]$.
- $X = [s, s-1, ..., 1, 0, 1, ..., N-1]$.

The optimal answer is the shorter of these two sequences.

1.2 Subtask 2

We observe that there always exists a rebalancing strategy with a distance $\leq 2N-2$. In this subtask, the tree is a path and N is small, allowing us to enumerate all possible walks. There are at most $N \times 2^{2N-2}$ such walks.

To check if a station sequence X is valid, we greedily construct Y as follows: For each station i, let its visited steps be $t_1, t_2, ..., t_k$ in chronological order.

- At t_1, set $Y[t_1] = -A[i]$ (load all initial bikes onto the truck).
- At t_k, set $Y[t_k] = B[i]$ (unload the target number of bikes at the station).
- If $t_1 = t_k$, then $Y[t_1] = B[i] - A[i]$.
- For all other steps t_j, set $Y[t_j] = 0$.

This construction ensures that the number of bikes at station i never drops below zero. We then verify if the truck always maintains a non-negative bike count, which is equivalent to checking if every prefix sum of Y is non-positive. It takes $O(N)$ time to check if the sequence Y is valid.

By finding the valid strategy with the minimum travelling distance, the overall time complexity is $O(N^2 2^{2N})$.

1.2.1 Correctness proof

Observation 1.1. If the input tree is a path with stations $0, 1, ..., N-1$ in sequential order, there exists a rebalancing strategy with travelling distance $2N-2$.

Proof. Consider the strategy: The truck starts at station 0, and collects all bikes at $0, 1, ..., N-1$. At this point, all bikes are on the truck. The truck then travels back and unloads bikes at stations $N-1, N-2, ..., 0$ to ensure each station i has exactly $B[i]$ bikes. □

Observation 1.2. If there exists a rebalancing strategy with station sequence X, then there exists a rebalancing strategy (X, Y) where Y is constructed as described above.

Proof. Let Y be the sequence constructed as described above. We aim to show that Y is valid. Suppose (X, Y') is any arbitrary valid strategy.

For each step t, we want to show that $\sum_{t' \le t} Y[t'] \le \sum_{t' \le t} Y'[t'] \le 0$. We consider the contribution to the sum from each station j:

- If station j is not visited again after step t, then

$$\sum_{\substack{t' \le t \\ X[t']=j}} Y[t'] = \sum_{\substack{t' \le t \\ X[t']=j}} Y'[t'] = A[j] - B[j].$$

- If station j has not been visited at all by step j, then

$$\sum_{\substack{t' \le t \\ X[t']=j}} Y[t'] = \sum_{\substack{t' \le t \\ X[t']=j}} Y'[t'] = 0.$$

- If station j has been visited at least once and will be visited again later, then

$$\sum_{\substack{t' \le t \\ X[t']=j}} Y[t'] = -A[j] \le \sum_{\substack{t' \le t \\ X[t']=j}} Y'[t']$$

  because the bike count at the station $A[j] + \sum_{t' \le t} Y'[t']$ must be ≥ 0.

Summing over all stations j:

$$\sum_{t' \le t} Y[t'] = \sum_{j=0}^{N-1} \sum_{\substack{t' \le t \\ X[t']=j}} Y[t'] \le \sum_{j=0}^{N-1} \sum_{\substack{t' \le t \\ X[t']=j}} Y'[t'] = \sum_{t' \le t} Y'[t'] \le 0.$$

□

Thus, to check if a station sequence X can be valid, it suffices to verify the specific construction of Y.

1.3 Subtask 4

Unlike previous subtasks, the road network is no longer a path but a general tree. However, there is still only one station s with $A[s] > 0$. Similar to subtask 1, we must find the shortest walk starting at s that visits all *required* stations.

In this subtask, there may be stations where $A[i] = B[i]$. If a leaf station i satisfies $A[i] = B[i]$, it does not need to be visited. We can repeatedly remove such leaves until all remaining leaves have $A[i] \neq B[i]$. After this pruning process, all remaining stations in the tree must be visited at least once.

Suppose t is the final station in the walk.

- Each edge on the simple path from s to t must be traversed at least once.
- All other edges in the tree must be traversed at least twice.

Thus, a walk from s to t that visits all stations has a length of at least $2(N-1) - \text{dist}(s,t)$. ($N$ is the number of required stations here.) This walk can be generated via DFS:

- Perform a DFS starting from s.
- For each station, treverse subtrees that do not contain the final station t first.
- Traverse the subtree containing t last.

The walk concludes immediately upon reaching t for the last time, achieving the lower bound distance. To minimize the total distance, we simply choose t as the station farthest from s in the pruned tree.

The time complexity is $O(N)$.

1.4 Subtask 3

1.4.1 Incorrect Approach

Following the ideas from subtask 1 and subtask 4, an intuitive approach is to find a starting station s and a final station t such that the strategy traverses edges on the s-t simple path exactly once and all other edges exactly twice. This would result in a travelling distance $2(N-1) - \text{dist}(s,t)$. (Note: This is incorrect, but we will explain this idea first.)

The question is which (s,t) pairs can support such a strategy. To identify these, we orient every edge according to the required flow of bikes across the edge. For every edge (u,v), let S_u and S_v be the net balance (the sum of $A[i] - B[i]$) of the two components separated by the edge, respectively. Since the total number of bikes is conserved, $S_u = -S_v$. We can orient the edge based on this balance:

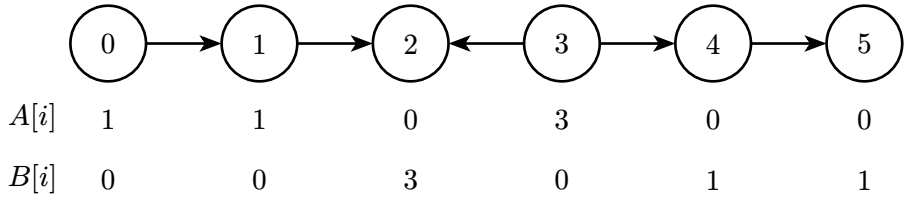
- $S_u > 0$: Bikes must move from u's side to v's side, so the orientation is $u \to v$.
- $S_u < 0$: Bikes must move from v's side to u's side, so the orientation is $u \leftarrow v$.
- $S_u = 0$: There is no net transfer required across the edge. The orientation can be in either direction; we assume it is oriented away from s.

Obviously, if an edge $u \leftarrow v$ has an orientation toward s, a valid strategy cannot traverse it only once; otherwise, we would be unable to move bikes from v's side to u's side. Therefore, to traverse edges on the s-t simple path only once, all edges on that path must be oriented toward t. We would then seek the longest directed path and let its endpoints be the starting station s and the final station t. Since this is the longest directed path, s has no incoming edges and t has no outgoing edges among all incident edges.

We can then construct a strategy. Without loss of generality, assume that $s < t$.

- The strategy starts at s. First, it collects all bikes from stations $s, s-1, ..., 0$, and unloads bikes at $0, 1, ..., s-1$ to reach their target $B[i]$. This is valid because $\sum_{i=0}^{s}(A[i] - B[i]) \geq 0$ due to the orientation $s \to s+1$.
- Then, for each $i = s, s+1, ..., t-1$, it loads or unloads bikes to achieve $B[i]$. This is valid because $\sum_{j=0}^{i}(A[j] - B[j]) \geq 0$ due to the orientation $i \to i+1$.
- Finally, it collects all bikes at stations $t, t+1, ..., N-1$ and unloads bikes at $N-1, N-2, ..., t$. This is valid because $\sum_{i=0}^{N-1}(A[i] - B[i]) = 0$.

Although this strategy is valid, **it is not the optimal solution**. Consider the following counterexample:



The approach above might identify $(s,t) = (0,2)$ or $(s,t) = (3,5)$, both of which have a travelling distance of 8. However, there is a better solution:

$$X = [0,1,2,3,2,3,4,5], \quad Y = [-1,-1,0,-3,3,0,1,1].$$

This strategy has a travelling distance of 7.

1.4.2 Correct Approach

The key insight is that edges on the s-t simple path **can** be oriented toward s, but such edges must be traversed three times. We assign weights to the edges: if an edge is oriented away from s, its weight is 1; if it is oriented toward s, its weight is -1. Let $w(s,t)$ be the total weight of edges on the s-t simple path.

- Every edge not on the s-t simple path must be traversed at least twice.
- Every edge on the s-t simple path oriented away from s must be traversed at least once.
- Every edge on the s-t simple path oriented toward s must be traversed at least three times.

Thus, a strategy from s to t has a travelling distance of at least $2(N-1) - w(s,t)$. Our goal is to find the (s,t) pair that maximizes $w(s,t)$. By the maximality, s will have no incoming edges and t will have no outgoing edges among all incident edges. We can then construct a walk that achieves the lower bound. Without loss of generality, assume $s < t$.

We partition the stations on the s-t simple path (including s and t) into groups $P_1, P_2, ..., P_k$ in order from s to t. Each group P_i contains $s_i, s_i+1, ..., t_i$ (where $s_i \le t_i$), defined such that:

- Within every P_i, all edges are oriented toward s.
- For every $1 \le i < k$, the edge (t_i, s_{i+1}) is oriented away from s.

That is, continuous segments of stations connected by edges oriented toward s form groups. We complete the rebalancing for each group sequentially from $i = 1$ to k. First, the truck collects all bikes at stations $s, s-1, ..., 0$ and unloads them at $0, 1, ..., s-1$. This is valid because $\sum_{i=0}^{s}(A[i] - B[i]) \ge 0$ due to the orientation $s \to s+1$. The truck then enters the first group.

For the i-th group, the truck first moves forward to collect all bikes at stations $s_i, s_i+1, ..., t_i$, then moves back to s_i without loading or unloading bikes, and finally moves to t_i and unloads bikes to stations $s_i, s_i+1, ..., t_i$. This is valid because after collecting bikes at t_i, the truck carries $\sum_{j=0}^{t_i} A[j] - \sum_{j=0}^{s_i-1} B[j]$ bikes. By our grouping method, $\sum_{j=0}^{t_i}(A[j] - B[j]) \ge 0$ ensures the truck bike count never drops below zero.

After the last group is processed, the truck collects all bikes at $t+1, t+2, ..., N-1$ and unloads bikes at $N-1, N-2, ..., t+1$.

In conclusion, (X, Y) is a valid strategy with a travelling distance of $2(N-1) - \max w(s,t)$. For this subtask, $\max w(s,t)$ can be found by assigning weights to edges (there are two cases: orienting flexible edges either right (if $s < t$) or left (if $t < s$)) and computing the longest weighted path. This can be implemented in $O(N)$ time.

1.5 Subtask 5, 6

The main idea is similar to subtask 3. First, we remove all leaves where $A[i] = B[i]$ as described in subtask 4, and assume N represents the number of remaining stations. We then enumerate the

starting station s and assign edge weights based on the directions of edges. (Recall that the directions of edges with flexible orientations are determined by s.) We choose the station t that maximizes the weighted path sum $w(s,t)$ as the final station. The minimum travelling distance is thus $2(N-1)-w(s,t)$.

We construct a strategy to achieve the distance bound. Similar to subtask 3, we partition the stations on the s-t simple path into groups $P_1, P_2, ..., P_k$ using the same logic. However, because the road network is now a general tree, we must handle subtrees branching off from the s-t path.

Consider the tree rooted at s. For each station i, let S_i be the sum of $A[j]-B[j]$ over stations j in the subtree rooted at i. The edge connecting station i and its parent is oriented upward if $S_i > 0$ and downward otherwise.

Observation 1.3. If the truck enters a subtree rooted at v carrying at least $\max(-S_v, 0)$ bikes, there exists a strategy to complete the rebalancing of all stations in that subtree with a travelling distance of $2(N_v-1)$, starting and ending at v, where N_v is the size of the subtree.

Proof. We prove this by induction. Suppose the truck is at station v and carries exactly $\max(-S_v, 0)$ bikes. First, the truck collects $A[v]$ bikes at station v. For every child c of v such that $S_c \geq 0$, since the truck carries at least $0 \geq -S_c$ bikes, we can complete rebalancing of the subtree rooted at c in $2(N_c-1)+2 = 2N_c$ steps and return to v. After rebalancing these subtrees in arbitrary order, the truck carries

$$\max(-S_v,0) + A[v] + \sum_{\substack{\text{child } c \\ S_c \geq 0}} S_c = \max(-S_v,0) + S_v + B[v] - \sum_{\substack{\text{child } c \\ S_c < 0}} S_c \geq B[v] + \sum_{\substack{\text{child } c \\ S_c < 0}} |S_c|$$

bikes. For every child c with $S_c < 0$, the truck now carries at least $-S_c$ bikes, allowing us to complete the rebalancing of those subtrees and return to v. Finally, we unload $B[v]$ bikes at station v. The total distance is $\sum_{\text{child } c} 2N_c = 2(N_v-1)$. □

In short, we rebalance subtrees rooted at children c with non-negative S_c first, followed by those with negative S_c.

The truck first enters the first group P_1. For each group P_i with members $v_1, v_2, ..., v_\ell$ (where $v_1 = s_i$ and $v_\ell = t_i$), s_i is an ancestor of t_i. Before entering P_i, the truck has completed rebalancing for all stations outside the subtree of s_i and carries exactly $-S_{s_i}$ bikes. ($-S_{s_i} \geq 0$ by the orientation $t_{i-1} \to s_i$ for all $s_i \neq s$, and $S_{s_1} = 0$.)

- The truck moves from s_i to t_i. For simplicity, we do not load bikes at intermediate stations $v_1, ..., v_\ell$ yet.
- The truck moves from t_i back to s_i. At each station v_j along the way:
  - The truck now carries $-S_{s_i} + S_{v_{j+1}} - S_{s_{i+1}}$ bikes. $S_u = 0$ if u does not exist. There are $-S_{s_i}$ bikes from the previous groups, and $S_{v_{j+1}} - S_{s_{i+1}}$ from previously processed stations in this group.
  - Load $A[v_j]$ bikes.
  - Rebalance all branching subtrees rooted at children c such that c is not on the s-t simple path and $S_c \geq 0$. After this, the truck carries

$$\underbrace{-S_{s_i}}_{\geq -S_{v_j}} + S_{v_{j+1}} - \underbrace{S_{s_{i+1}}}_{\leq 0} + A[v_j] + \sum_{\substack{\text{child } c \notin P \\ S_c \geq 0}} S_c \geq -S_{v_j} + S_{v_j} + B[v_j] - \sum_{\substack{\text{child } c \notin P \\ S_c < 0}} S_c \geq B[v_j] + \sum_{\substack{\text{child } c \notin P \\ S_c < 0}} -S_c$$

bikes, where P is the s-t simple path, and $S_{v_{j+1}} = S_{s_{i+1}}$ if v_{j+1} does not exist; $S_{s_{i+1}} = 0$ if s_{i+1} does not exist.

- Rebalance all branching subtrees rooted at children c such that c is not on the s-t simple path and $S_c < 0$.
- Finally, unload the required $B[v_j]$ bikes at station v_j.

- Move to station t_i.

The travelling distance is

$$2(N-1) - \text{dist}(s,t) + \sum_{i=1}^{k}(|P_i| - 1) = 2(N-1) - w(s,t).$$

We can find the maximum $w(s,t)$ by enumerating each possible s and computing the weighted path sums for every t. This results in an $O(N^2)$ implementation, which is sufficient for Subtask 6. Subtask 5 allows for a less efficient $O(N^3)$ approach.

1.6 Subtask 7

We can improve the time complexity to $O(N)$ using dynamic programming. First, root the tree at an arbitrary station r. For every station $v \neq r$, we compute $dp_{\uparrow}[v]$ and $dp_{\downarrow}[v]$. Here, $dp_{\uparrow}[v]$ represents the maximum path weight from ***v*'s parent** to a descendant of v, assuming all edges with flexible directions are oriented upward. $dp_{\downarrow}[v]$ is defined similarly, but edges with flexible directions are oriented downward.

These values can be computed efficiently using bottom-up transitions. The maximum weighted path $w(s,t)$ can be found by considering each station v as the lowest common ancestor of s and t. $w(s,t)$ is the maximum value among

$$\max_{v} \max_{\substack{\text{child } c_1, c_2 \text{ of } v \\ c_1 \neq c_2}} \{dp_{\uparrow}[c_1] + dp_{\downarrow}[c_2]\}, \quad \max_{v \neq r}\{dp_{\uparrow}[v]\}, \quad \max_{v \neq r}\{dp_{\downarrow}[v]\}.$$

By backtracking through the DP states or storing the endpoints during the transitions, we can identify the specific stations s and t that yield the maximum weight. Finally, we construct the rebalancing strategy using the method described in the previous subtask. The total time complexity is $O(N)$.

2 Navigation

2.1 Solution

In the task analysis, we refer to the network as a graph (which is a tree), each node being a vertex, and each cable being an edge. We call the power stations special nodes and others plain nodes, and the cable types as colors of the edges.

From now on, we only consider deterministic strategy, which means the robot should be able to correctly determine the answer for every plain nodes and for any permutation.

For each node u, we refer to the subtree of each edge as the connected component not containing u when the edge is removed.

2.2 Basic Observations

From the problem, there are several properties and simple observations we can make:

- The number of special nodes with distance decreased after following an edge, is equivalent to the number of special nodes in that subtree.
- If two edges adjacent to a node have the same color, the answer of the edge must be the same, because we cannot distinguish the edges.
- We **don't** need to find the answer for special nodes. This property is actually not a derived one, but stated in the statement twice (three times if you count the one in the examples), but it remains relevant for developing many of the ideas.

2.3 Subtask 1: $N = 7$

With the observation above, it should be very clear that this subtask is a "sanity check" subtask.

When $K = 6$ and $N = 7$, it is certain that the only possible query is the remaining node. Thus, it is possible to directly encode the answer; the naive method uses type $0, 1, ..., 6$, giving $S = 7$.

Since at most one number is greater than 3, we can decrease the largest number to 3. For any query such that the color indices does not sum to $K = 6$, we increase the largest entry until it satisfies the condition, and report that as the answer. This method gives $S = 4$.

2.4 Subtask 2: the network forms a chain

No queries is made at the endpoints and the special nodes, so we can partition the chain into disjoint paths by the special nodes. For each path, the answer multiset is either $\{0, 6\}, \{1, 5\}, \{2, 4\}$, or $\{3, 3\}$.

When the answer is $\{3, 3\}$, we can always assign *any* same color to both edges. This is because all other paths should never have two same color edges incident to a node, as that makes the two edges indistinguishable.

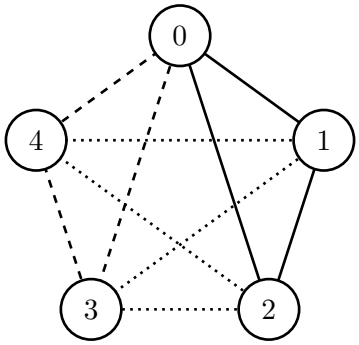
For the remaining paths, for any node that is adjacent to edges with color c_1 and c_2, respectively, we add a directed edge from $c_1 \to c_2$ if the answer to edge with color c_1 is bigger, and $c_1 \leftarrow c_2$ otherwise.

The resulting graph must have these properties:

- No self loop exists.
- For any pair of distinct vertices u, v, at most one of $u \to v$ and $v \to u$ exists.

Suppose that path is colored by $c_1, c_2, ..., c_k$, because k can be made almost arbitrarily large, it follows that the edges added must form a cycle. Combining the observation above, in order to answer $\{0, 6\}, \{1, 5\}, \text{and } \{2, 4\}$, it remains to solve the following problem: what is the minimum S such that the complete graph K_S can be partitioned into 3 edge-disjoint simple cycles?

The answer is $S = 5$, as follows:



Surprisingly, this construction is unique up to permuting the colors, and it is rather straightforward when the required observation are known.

2.5 Partial Approaches to Subtask 3, 4, and 5

Subtask 3, 4, and 5 only differs by the constraint on the degree of the internal nodes. From now on, we will instead describe different coloring schemes and their related observations or idea.

These solutions are all partial solutions, but their various observations will hint and even give some intuition of how an optimal encoding might look like.

2.5.1 $S = 64$ for subtask 3 and 4

If the coloring is separate for two directions of each edge, the task becomes very easy since we basically can encode the answer. However, the main challenge is we don't know the direction. Thus, one natural way to solve it is by first rooting the tree, encode the rooted information, then encode something for the node to find its parent.

When the minimum degree is at least 3, it is possible to encode the parent information with two colors. Specifically, we color each edge by their distance to the root modulo 2. Thus, the parent edge will always be different from other edges, and it is unambiguous when each internal node has degree of at least 3.

Based on this, we can recover the answer with $S = 2^6 = 64$: we simply root at every special node and use a bit to encode the parent information. To recover, we find each parent and add 1 to the edge for each of the 6 bits.

2.5.2 $S = 12$ for subtask 3 and 4

For subtask 3 and 4, we can extend the rooting idea to root for just one special node and encode the remaining 5 special nodes separately. Since the root information gives us orientation, we can encode the number of special nodes in each subtree on the edge. Thus, when recovering each node, we first identify the parent edge, and invert the number of nodes in the parent edge by $x \mapsto 6 - x$. This solution works because a special node is never queried, so we won't have the off-by-one issue.

This approach may yield $S = 14$ if the participant did not realize we only need 0 to 5 instead of 0 to 6.

2.5.3 $S = 18$ for the general case

Extending the idea above, the issue for the previous approach is because we cannot identify which is the parent edge for internal node with degree 2. However, inspired from subtask 2, it is possible to use 3 colors to find the parent edge.

We root at one special node similarly, and instead of modulo 2, we color each edge by their distance to the root modulo 3. When recovering, if we know the missing color is c, then the parent edge is the one with color $+1 (\mathrm{mod}\, 3)$.

Similarly, this approach may yield $S = 21$ if the participant did not realize we only need 0 to 5 instead of 0 to 6.

2.5.4 $S = 8$ for subtask 3 and 4

We root at a special node in the previous approach, but we actually did not benefit from any of those since identify the root is still possible.

Thus, we can instead root at a *weighted centroid* of the tree. That is, we pick some node such that every subtree has at most 3 special nodes in each subtree. Finding one such node is similar to finding a ordinary centroid of the tree, and the proof of such node always existing is similar to the original one.

Combining with the previous approach, we achieves $S = 8$ because the number of special nodes in each subtree can only have $0, 1, 2$, and 3.

2.5.5 $S = 7$ for subtask 3 and 4

It follows that we don't need to identify the parent if the answer is 3, because $3 = 6 - 3$. Thus, we don't perform the two-coloring for this case, saving one color.

2.5.6 $S = 10$ for the general case

The previous solution naturally generalizes to the general case by replacing the two-coloring with three-coloring. Similarly, naively this approach takes $S = 12$, but the case for answer equal to 3 is similar, saving two colors.

2.6 Full Solution to Subtask 3 and 4

After these partial attempts, we have some good tricks to perform and we should have a sense of why are we using so many colors. In this subsection, we will introduce two approaches.

2.6.1 Approach 1

The first approach is by trying to relax the rooting condition. For each edge, we encode the smaller number of special nodes in each subtree. When recovering, if the neighboring edge sums to 6, then we know that each edge is at the correct orientation; however, if it is not, the answer multiset must be one of $\{4, 1, 0, ...\}$, $\{5, 2, 0, ...\}$, and $\{6, 0, ...\}$. In addition, there is one more related case: $\{4, 1, 1, 0, ...\}$. We will refer to the cases by 4-2, 4-1-1, 5-1, and 6-0.

These cases do form a really good structure because of the tree property: the 4-2 and the 5-1 cases forms a chain, connected by 4-1-1, and all other branches are the case 6-0. Naturally, we would want to apply the solution for subtask 2.

But how can we try to obtain one such encoding? There are two major constraints that give us a really good idea:

1. These special encoding should never produce a case where the edge colors sum to 6.
2. Because finding two 3-cycles in a graph requires 5 nodes, with $S = 4$, we must use the two extra degree of each internal node to also represent some information.

It is natural to write a bruteforce program to find a set that satisfies the constraints, but because the restrictions are tight, it is actually feasible to found a solution by hand.

The following section describes one such scheme. We use k-subtree to refer to all edges whose subtree has answer equal to k.

- Case 4-2: Always color the 0-subtree with color 3.
  - If the parent edge has color 2, color the 2-subtree with 1.
  - If the parent edge has color 1, color the 2-subtree with 0.
  - If the parent edge has color 0, color the 2-subtree with 2.
- Case 4-1-1: Always color the 0-subtree with color 3.
  - If the parent edge has color 2, color the 1-subtrees with 0.
  - If the parent edge has color 1, color the 1-subtrees with 0.
  - If the parent edge has color 0, color the 1-subtrees with 1.
- Case 5-2: Always color the 0-subtree with color 2.
  - If the parent edge has color 1, color the 1-subtree with 0.
  - If the parent edge has color 0, color the 1-subtree with 3.
  - If the parent edge has color 3, color the 1-subtree with 1.
- Case 6-0:
  - If the parent edge has color 3, color the remaining edges with color 2.
  - Otherwise, color the remaining edges with color 3.

One extra note of handling these cases: since it is possible to encounter special nodes in these constructions, one way to avoid the cases is to treat the special node belonging to the subtree of the parent edge. By giving default values for each case (for example, deliberately write `else` instead of `else if` for the final case as default fallback), this case would be easily circumvented.

For partial scores, for example $S = 5$ or 6, it is possible to use some extra color to mark these cases, and thus it is very constructible once the relevant observations have been made.

2.6.2 Approach 2

The second approach is very different from the previous approaches. The idea is to use a similar idea to the orienting idea. Since there are at most 4 different answers (0, 1, 2, 3), we try to use $S = 5$ and use the missing color to find the correct cyclic shift.

The encoding scheme is as follows. For each possible integer partition, we assign a canonical encoding, which will be a prefix of the colors. Thus, we can always recover the canonical encoding with the missing colors.

- If the answer is $\{6, 0, ..., 0\}$, the canonical encoding is $\{1, 0, ..., 0\}$.
- If the answer is $\{5, 1, 0, ..., 0\}$, the canonical encoding is $\{2, 1, 0, ..., 0\}$.
- If the answer is $\{4, 2, 0, ..., 0\}$, the canonical encoding is $\{2, 0, 1, ..., 1\}$.

- If the answer is $\{4, 1, 1, 0, ..., 0\}$, the canonical encoding is $\{2, 1, 1, 0, ..., 0\}$.
- If the answer is $\{3, 3, 0, ..., 0\}$, the canonical encoding is $\{1, 1, 0, ..., 0\}$.
- If the answer is $\{3, 2, 1, 0, ..., 0\}$, the canonical encoding is $\{3, 2, 1, ..., 0\}$.
- If the answer is $\{3, 1, 1, 1, 0, ..., 0\}$, the canonical encoding is $\{2, 1, 1, 1, 0, ..., 0\}$.
- If the answer is $\{2, 2, 2, 0, ..., 0\}$, the canonical encoding is $\{1, 1, 1, 0, ..., 0\}$.
- If the answer is $\{2, 2, 1, 1, 0, ..., 0\}$, the canonical encoding is $\{2, 2, 1, 1, 0, ..., 0\}$.
- If the answer is $\{2, 1, 1, 1, 1, 0, ..., 0\}$, the canonical encoding is $\{2, 1, 1, 1, 1, 0, ..., 0\}$.
- If the answer is $\{1, 1, 1, 1, 1, 1, 0, ..., 0\}$, the canonical encoding is $\{1, 1, 1, 1, 1, 1, 0, ..., 0\}$.

Note that the second case and the third case requires 4 edges to distinguish, which fits within this constraint.

This gives uses $S = 5$. To reduce one color, observe that the only case requiring all 4 colors and 1 extra is this specific case of canonical encoding of $\{3, 2, 1, 0, ..., 0\}$ being $\{3, 2, 1, ..., 0\}$. If only one such node exists, we can take it as the root. Otherwise, there must be at most two. Since the path connecting them will all have the $\{3, 3, 0, ..., 0\}$ case, it is always possible to encode them with the exact canonical encoding.

2.7 Full Solution to Subtask 5

The complete solution is very similar to that of the Approach 1 of the previous subtask. Because of subtask 2, it is clear that the scheme should exactly follow the unique cycle partition of K_5.

If the participant realizes this for both solutions, writing a bruteforce program is very beneficial and should run sufficiently fast due to the known constraints.

Nevertheless, we show one possible scheme. This scheme is constructed by hand and thus, practically, participants are not required to write a solver for this.

- Case 4-2:
  - If the parent edge has color 2, color the 2-subtree with 1 and the 0-subtrees with color 0.
  - If the parent edge has color 1, color the 2-subtree with 0 and the 0-subtrees with color 4.
  - If the parent edge has color 0, color the 2-subtree with 2 and the 0-subtrees with color 4.
- Case 4-1-1: Always color the 0-subtree with color 4.
  - If the parent edge has color 2, color the 1-subtrees with 1.
  - If the parent edge has color 1, color the 1-subtrees with 3.
  - If the parent edge has color 0, color the 1-subtrees with 1.
- Case 5-2:
  - If the parent edge has color 1, color the 1-subtree with 3 and the 0-subtrees with color 0.
  - If the parent edge has color 3, color the 1-subtree with 4 and the 0-subtrees with color 2.
  - If the parent edge has color 4, color the 1-subtree with 1 and the 0-subtrees with color 2.
- Case 6-0:
  - If the parent edge has color 0, color the remaining edges with color 4.
  - If the parent edge has color 4, color the remaining edges with color 2.
  - If the parent edge has color 2, color the remaining edges with color 3.
  - If the parent edge has color 3, color the remaining edges with color 0.

  This solves the general case with $S = 5$.

3 Scallion Pancake Party

We describe two different solutions to this task.

3.1 Solution 1

For convenience, we will call the guest in room i "guest i".

The core idea is to partition the bags into N distinct sets, with each set assigned to a specific guest according to their room number. This assignment allows each guest to relay information to the guest outside through their respective sets. However, implementing this approach effectively requires addressing several technical challenges, which we explore in the following subsections.

3.1.1 Subtask 2

Starting with Subtask 2, let's explore how the guest outside can correctly recover the permutation P.

Since every guest knows their room number, we can naturally partition the bags into N sets, assigning one set to each guest based on their room number. This allows each guest to communicate information to the outside guest by selectively eating bags only from their assigned set.

One partitioning strategy is to assign the $(i+1)$-th occurrence of every flavor to guest i. By counting the occurrences of each flavor, all guests can easily reach a consensus on which bags belong to them.

Furthermore, as each guest now has exactly one bag of each flavor in their assigned set, guest i can simply eat all bags except the one with flavor $P[i]$.

Consequently, the guest outside can identify which set each remaining bag belongs to and determine P based on the remaining flavor.

3.1.2 Subtask 3

In this subtask, we no longer have access to the room numbers. If we wish to apply the "partition and eat all but one" strategy from subtask 2, each guest must first determine their own room number by observing what previous guests have eaten.

In subtask 3, the bags follow a convenient structure: their flavors are a concatenation of N permutations. Thus, we can partition the bags by their indices such that the i-th bag is assigned to the $\left\lfloor \frac{i}{N} \right\rfloor$-th set.

But how can a guest identify their room number? Recall the strategy: the guest in room i eats all bags in their set except for the one with flavor $P[i]$. In an optimistic scenario, guest i sees that bags with indices in the range $[(i-1) \cdot N, i \cdot N)$ have already been eaten, allowing them to infer that their room number is **at least** i.

Specifically, if the flavor of bag $i \cdot N$ is not $P[i]$, guest i can eat it. Subsequent guests will then observe this and realize their room number must be greater than i. A problem arises only when the flavor of bag $i \cdot N$ is exactly $P[i]$.

Let's simulate this case. Assume this first occurs for room i. Guest $i+1$ will find that bag $i \cdot N$ remains uneaten and mistakenly conclude that their room number is i. Consequently, bag $i \cdot N$ will be eaten unexpectedly by guest $i+1$. However, once guest i eats bag $i \cdot N + 1$, guest $i+1$ will notice this and realize their mistake. Furthermore, since bag $i \cdot N$ was already eaten by guest $i+1$, all

subsequent guests can correctly recognize that their room number is at least $i+1$. By induction, it can be proven that this process succeeds, with one exception: all bags belonging to the i-th set are eaten **if and only if** the flavor of bag $i \cdot N$ is exactly $P[i]$.

Finally, we recover the permutation P as the outside guest. If any bag remains uneaten in the i-th set, $P[i]$ is determined by its flavor. For sets where all bags were eaten, we use the "if and only if" condition: the flavor of bag $i \cdot N$ must have been $P[i]$. Thus, all values of P can be successfully recovered.

3.1.3 Subtask 4

We continue applying the "partition and eat all but one" strategy from subtask 2. With $K = N$, we have almost sufficient capacity to notify all guests of their room numbers using only the first bag. Specifically, each guest eats one slice from the first bag; thus, guest i will observe that exactly $i-1$ slices have already been eaten, identifying their room number as i.

A problem arises if a guest's assigned flavor $P[i]$ matches the flavor of this first bag. Since each flavor appears exactly once per permutation, this collision occurs exactly once. Suppose this happens for the guest j. All guests after them will observe one fewer eaten slice than expected, leading them to underestimate their room number by one (mistaking it for $j, j+1, ...$ instead of $j+1, j+2, ...$), while guests 1 through j will identify their numbers correctly.

For convenience, we ignore any subsequent bags that share the same flavor as the first bag. These bags are reserved solely for transmitting room number information; no further action is required when a guest encounters them (though they are welcome to eat from them if they are particularly hungry).

We then partition the remaining bags into N sets based on flavor occurrences, as in subtask 2. For guests in rooms $i < j$, the strategy works perfectly. However, the guest in room j must notify subsequent guests of the offset error. Since $K \geq 2$, the guest in room j can eat exactly one slice from the first bag of their assigned set. Subsequent guests will detect this "missing slice" signal and, upon receiving it, increment their inferred room number by one to correct the mistake.

Finally, the outside guest can recover all values of P except for $P[j]$. Since P is a permutation, $P[j]$ can then be uniquely determined by the process of elimination.

3.1.4 Subtask 6

We skip subtask 5 as it is intentionally left for some suboptimal solutions.

We now generalize the strategy from subtask 3. Once again, we partition the bags into N sets based on their occurrences, where set i consists of the $(i+1)$-th occurrence of every flavor. Each guest continues to follow the established strategy: eat all bags belonging to their assigned set, except for one.

We can generalize the observation made in subtask 3 as follows:

Observation 3.1. If a guest observes that the i-th occurrence of any flavor has been completely eaten, they can infer that their room number is at least i.

However, relying solely on this observation leads to problems, e.g., when the very first bag of set i happens to have the flavor $P[i]$.

The primary issue is that guest $i+1$ will see this uneaten bag and mistake their room number for i, leading them to unexpectedly eat the bag in set i with flavor $P[i]$. While guest i can detect this mistake the moment they encounter the bag, unlike in subtask 3, they cannot always notify guest $i+1$ simply by eating the entirety of the next bag. Nevertheless, we claim that a robust strategy for the guest outside can resolve this ambiguity by examining the next incoming uneaten bag according to these three cases:

1. If the bag belongs to set i (i.e., it is another flavor's $(i+1)$-th occurrence): Guest i can directly eat the entire bag. Guest $i+1$ will see this and immediately realize their mistake.
2. If the bag is the next occurrence of the same flavor: This bag will be eaten by guest $i+2$. This cascading effect is generally acceptable, though it may result in the final occurrence of this flavor remaining uneaten. We will explain how the outside guest resolves this ambiguity later.
3. If the bag belongs to a previous set (i.e., it is an occurrence $< i+1$): This bag was intentionally left uneaten to notify the outside guest of a previous P value, just simply ignore it.
   - If you do not realize that simply ignoring Case 3 is sufficient for the guest outside, you might be tempted to implement a more complex signaling system. For instance, one could utilize the $K \geq 3$ condition to notify guest $i+1$ of their mistake by eating a single slice from the bag, and guest $i+1$ will need to eat an additional slice to cancel the signal.

Under this strategy, we can establish the following:

Observation 3.2. Let B_i be the bag of flavor F belonging to set i, where F is a flavor not equal to any of $P[0], P[1], ..., P[i-1]$. Guest i will always successfully deduce their correct room number by the time they encounter bag B_i.

Proof. We prove this by contradiction and induction. Assume the statement holds for guests 0 through $i-1$. Suppose guest i still does not know their room number when they reach bag B_i. This implies that the previous occurrence of flavor F was not eaten by guest $i-1$; otherwise, by Observation 3.1, guest i would have known their room number.

However, this is a contradiction to the induction hypothesis, as flavor F is not equal to any of $P[0], P[1], ..., P[i-2]$, and guest $i-1$ should eat it according to our strategy. □

Finally, we describe the strategy for the outside guest to recover the permutation P. Assume we have successfully recovered $P[0]$ through $P[i-1]$, and we now aim to deduce $P[i]$:

- If some bags in set i remain uneaten: By Observation 3.2, guest i eventually knew their room number and ate the remaining bags in set i. Thus, the flavor of the last uneaten bag in set i must be exactly $P[i]$.
- If all bags in set i are eaten: The flavor of the first bag in set i that does not belong to $\{P[0], ..., P[i-1]\}$ must be $P[i]$.
  - If this first available flavor were not $P[i]$, guest i would have recognized their room number upon seeing it (since it avoids all previous P values) and would have eaten it, leaving the actual $P[i]$ bag uneaten. Furthermore, all subsequent guests would correctly deduce that their room numbers are strictly greater than i, ensuring none of them would mistakenly eat this remaining $P[i]$ bag in set i. Thus, the $P[i]$ bag would definitely remain uneaten, resulting in a contradiction.

3.2 Solution 2

Let $A_{f,i}$ denote the index of the $(i+1)$-th bag that contains flavor f.

The core idea is to force a guest with forbidden flavor P_i to only eat pancakes from bags with flavor $(P_i+1) \bmod N$.

Unless specified, eating a bag means eating all remaining slices from the bag.

3.2.1 Subtask 1

Guest i always eats bag $A_{(P_i+1) \bmod N,0}$. They only eat bag $A_{(P_i+1) \bmod N,1}$ if $A_{P_i,0} < A_{(P_i+1) \bmod N,0}$ and bag $A_{P_i,0}$ has not been eaten.

For each guest i, the condition $A_{P_i,0} < A_{(P_i+1) \bmod N,0}$ can be easily checked the moment they receive bag $A_{(P_i+1) \bmod N,0}$. They simply check whether a bag with flavor P_i has appeared before. Since $A_{(P_i+1) \bmod N,0} < A_{(P_i+1) \bmod N,1}$, they will know whether they should eat $A_{(P_i+1) \bmod N,1}$ by the time they receive it.

For example, suppose $F = [1, 0, 0, 1]$.

- If $P = [0, 1]$, then guest 0 only eats bag 0 and guest 1 only eats bag 1.
- If $P = [1, 0]$, then guest 0 eats bags $1, 2$ and guest 1 only eats bag 0.

For the guests outside, they can clearly tell which of $A_{0,0}, A_{1,0}$ comes first. Suppose $A_{f,0} < A_{1-f,0}$ for some $f = 0$ or 1.

Observe that bag $A_{f,1}$ will never be eaten. The guests will guess P based on whether bag $A_{1-f,1}$ is empty. If bag $A_{1-f,1}$ is empty, then $P = [1-f, f]$. Otherwise, $P = [f, 1-f]$.

3.2.2 Subtask 2

In this subtask, each guest i knows the value of i.

Guest i simply eats bag $A_{(P_i+1) \bmod N,i}$.

For the guests outside, they will see exactly N empty bags. Each empty bag $A_{f,i}$ tells them the value of $P[i]$, which is just $(f + N - 1) \bmod N$.

3.2.3 Subtask 3

In this subtask, $A_{0,0}, A_{1,0}, ..., A(N-1, 0)$ are the first N bags.

For the first N bags, guest i only eats $A_{(i+1) \bmod N,0}$.

After the first N bags, guest i will see exactly $i+1$ empty bags. So each guest can learn their room index once they reach this point.

So the strategy for the remaining $N(N-1)$ bags are:

- For each $0 \le i \le N-2$, guest i eats bag $A_{(P_i+1) \bmod N,i+1}$.
- Guest $N-1$ does nothing.

For the guests outside, they can determine P based on empty bags from the last $N(N-1)$ bags.

Each empty bag $A_{f,i}, i \ge 1$ tells them the value of $P[i-1]$, which is just $(f + N - 1) \bmod N$.

There will be exactly one flavor f without empty bags. Which gives $P[N-1] = (f + N - 1) \bmod N$. Alternatively, since there will be exactly one value, $P[N-1]$, missing in P, it can also be determined using the filled values.

3.2.4 Subtask 6

We skip subtasks 4 and 5 and jump straight into the full solution.

Without extra info or specific bag sequence patterns, it seems quite hard for guests to first learn their room index and encode that information with later bags. To solve this issue, perhaps we can extend the idea for subtask 1.

First of all, each guest i will always eat bag $A_{(P_i+1) \bmod N,0}$.

For each guest i, Let e_{P_i} be the number of flavors f such that $A_{f,0} < A_{(P_i+1) \bmod N,0}$ and bag $A_{f,0}$ is empty. They can determine e_{P_i} once they receive bag $A_{(P_i+1) \bmod N,0}$.

Then, each guest i encodes e_{P_i} using $A_{(P_i+1) \bmod N,1}, A_{(P_i+1) \bmod N,2}, ..., A_{(P_i+1) \bmod N,N-1}$. More specifically, they eat bag $A_{(P_i+1) \bmod N,j}$ if and only if the j-th bit of e_{P_i} is 1. Since $e_{P_i} \leq N-1$, the number of bags is enough to encode the bit representation of e_{P_i}.

For the guests outside, they can easily obtain e_i by the binary representation of $A_{(i+1) \bmod N,1}, A_{(i+1) \bmod N,2}, ..., A_{(i+1) \bmod N,N-1}$. We describe how to reconstruct P using $e_0, e_1, ..., e_{N-1}$.

Let $f_0, f_1, ..., f_{N-1}$ be the order of flavors such that $A_{f_0,0} < A_{f_1,0} < ... < A_{f_{N-1},0}$. In other words, f_i is the $(i+1)$-th flavor that appeared for the first time in the sequence of bags.

Observe that for each $1 \leq i \leq N-1$, e_{f_i} is precisely the number of flavors f among $f_0, f_1, ..., f_{i-1}$ such that f appears earlier than f_i in P. This means we can do something like insertion sort to determine P.

More specifically, we start with $P = [f_0]$. Then, for each i from 1 to $N-1$, we insert f_i to P such that it becomes the $\left(e_{f_i} + 1\right)$-th element in P. In the end, we will have the correct P.

3.3 Remarks

- The score of the last subtask originally had partial scoring: Higher score is awarded if the total number of slices remaining is lower. We decided to remove the partial scoring since it was not an important part of the solution, and it barely changes the difficulty of the problem.
- There was a "harder" version of the task: Reduce the number of bags of each flavor to 6 instead of N. This cuts out solution 1 while solution 2 still passes since it requires just $\lceil \log_2 N \rceil + 1 \leq 6$ bits of information for each flavor. We ultimately decided to keep the current version so that the entire problem set wouldn't be too hard. (The problem set is already hard enough.)